

FAQ Assistant API

Project Documentation Phase Document

Item	Details
Project Name	FAQ Assistant API
Document	Documentation Phase
Version	1.0 (Draft)
Phase	Documentation (runs alongside Development, Testing and Deployment)
Status	For review

1. Introduction

Good documentation lets other people understand, use, maintain and extend the FAQ Assistant API without asking the original developers. This document explains what documents will be created, who they are for, how they will be written and reviewed, and when they will be delivered.

2. Documentation Objectives

- Explain clearly how client developers can integrate the API.
- Help admins manage FAQ content without developer support.
- Help new team members set up, understand and maintain the code.
- Keep a complete record of requirements, design, testing and decisions.
- Keep all documents accurate, consistent and up to date with the code.

3. Documentation Plan

Document	Audience	Content	Format
API Reference	Client developers	Endpoints, parameters, examples, error codes	Swagger / OpenAPI
README	All developers	Overview, setup, run and test steps	Markdown
Developer Guide	Team / maintainers	Architecture, structure, coding standards, workflows	PDF / Markdown
Admin Guide	Content admins	How to add, edit, delete FAQs and read unanswered questions	PDF
User / Integration Guide	Client app teams	Step-by-step integration with examples	PDF
Deployment Guide	DevOps / maintainers	Environment variables, Docker, release steps	PDF / Markdown
Test Documents	QA / owner	Test plan, cases, reports	PDF
Project Reports	Owner / guide	Planning, SRS, design and final report	PDF
Change Log	Everyone	Version history and changes	Markdown

4. API Reference Documentation

The API reference is generated automatically from the code (OpenAPI) and improved with descriptions and examples. Every endpoint will be documented with the following details:

- Purpose of the endpoint and who can use it.
- HTTP method, URL and required headers (for example the API key or token).
- Request parameters and body with data types and limits.
- Success response with a sample.

- All possible error responses with status codes and messages.
- A ready-to-run example (cURL and one programming language).

4.1 Sample Endpoint Entry

```

POST /ask
Purpose : Get the best answer for a user question
Auth    : API key in header X-API-Key: your_key

Request body
  question  string    required    1 to 500 characters

Success (200)
  answer, matched_question, confidence (0 to 1), faq_id

Errors
  400 Question is empty or too long
  401 Missing or invalid API key
  429 Rate limit exceeded

cURL example
curl -X POST https://api.example.com/ask \
  -H "X-API-Key: your_key" \
  -H "Content-Type: application/json" \
  -d '{"question": "How can I reset my password?"}'

```

A Postman collection with all endpoints and sample requests will also be provided so that developers can try the API immediately.

5. README Structure

The README is the first file anyone reads in the repository. It will follow this structure:

Section	What it Contains
Project title and summary	What the FAQ Assistant API does in two or three lines
Features	Main capabilities of the API
Tech stack	Language, framework, database, libraries
Getting started	Prerequisites, installation, environment variables, running the server
Usage	Basic request and response examples
Running tests	Commands to run unit and API tests
Project structure	Folder layout and purpose of each part
Deployment	Short steps or link to Deployment Guide
Contributing and license	Rules for contributing and license information

6. Guides

6.1 Admin Guide

- How to log in or use the admin key.
- How to add a new FAQ with category and tags.

- How to edit or remove an existing FAQ.
- How to review unanswered questions and turn them into new FAQs.
- How to read feedback and improve weak answers.
- Tips for writing clear questions and answers.

6.2 Integration Guide for Client Apps

- How to get an API key.
- Sending the first request and reading the response.
- Handling low-confidence answers and the fallback message.
- Sending user feedback (helpful or not helpful).
- Handling errors and rate limits properly.

6.3 Developer and Deployment Guide

- Architecture overview and module explanation.
- Local setup, environment variables and database migrations.
- Coding standards, branching and pull request rules.
- Docker build, deployment steps and rollback procedure.
- Monitoring, logs and common troubleshooting steps.

7. Code Documentation Standards

- Every public function and class has a docstring explaining purpose, inputs and outputs.
- Use type hints so that the code explains itself.
- Comments explain why something is done, not only what is done.
- Complex logic such as the matching engine has a short written explanation in the developer guide.
- Configuration options are listed with default values in the README or .env.example.

8. Tools Used

Purpose	Tool
Interactive API docs	Swagger UI / ReDoc (auto-generated by the framework)
API testing and sharing	Postman collection
Written documents	Markdown files in the repository, exported to PDF
Diagrams	Draw.io or Mermaid (architecture, flow, ER diagram)
Version control	Git and GitHub (documents are versioned with the code)
Docstring checks	Linters rules to warn about missing docstrings

9. Documentation Standards and Versioning

- Use simple, clear language and short sentences; avoid unexplained jargon.
- Use the same terms everywhere (for example always say FAQ, not question-answer pair).
- Every document has a title, version, date, author and status on the first page.
- Version numbers follow major.minor format, for example 1.0, 1.1, 2.0.
- Every release updates the change log with added, changed and fixed items.
- Documents are stored in the repository in a docs folder.

10. Review Process

Step	Activity	Done By
1	Write the first draft	Document owner
2	Technical review for accuracy	Developer / team lead
3	Usability review: can a new person follow the steps?	Tester or new team member
4	Correct comments and finalize	Document owner
5	Approve and publish	Project owner

11. Quality Checklist

- All endpoints are documented and examples actually work.
- Setup steps were tested on a clean machine.
- No spelling mistakes, broken links or outdated screenshots.
- Error codes and messages match the real API behaviour.
- Documents are consistent with the latest code version.

12. Schedule

Activity	When	Duration
Documentation plan and templates	Planning phase	2 days
README and code docstrings	During development	Ongoing
API reference and Postman collection	End of development	2 days
Test documents	During testing	Ongoing
Admin, integration and deployment guides	After testing	3 days
Review, correction and final release	Before deployment	2 days

13. Roles and Responsibilities

Role	Responsibility
Project Manager	Plans documents, tracks status, ensures reviews are done
Developers	Write code docs, README, API descriptions and developer guide
QA / Testers	Write test documents and check guides by following them
Technical Writer / Team Member	Write and format user, admin and deployment guides
Project Owner	Reviews and approves final documents

14. Risks and Mitigation

Risk	Mitigation
Documents become outdated after code changes	Update docs in the same pull request as the code
Documentation left until the end	Write in every phase and track it as a task
Examples in the docs do not work	Test every example before publishing
Different writing styles	Use common templates and a review step

15. Deliverables

- API reference (Swagger / OpenAPI) and Postman collection.
- README file and developer guide.
- Admin guide and integration guide.
- Deployment guide and change log.
- Complete set of phase documents: planning, requirements, design, development, testing.

16. Exit Criteria

- All planned documents are written, reviewed and approved.
- A new person can set up and run the project using only the documents.
- Documents match the released version of the API.

17. Next Phase

With documentation complete, the project moves to Phase 6 - Deployment, where the API is released, monitored and handed over for maintenance.